

Optimization Techniques in Neural Networks

Adam, RMSprop, Learning Rate Scheduling, and Batch Normalization

Moin Mostakim

BRAC University

Outline

- 1 Introduction
- 2 Adaptive Optimization Methods
- 3 Introduction to Adam
- 4 Mathematical Formulation
- 5 Step-by-Step Example
- 6 Summary
 - RMSprop
- 7 Learning Rate Scheduling
- 8 Batch Normalization
- 9 Summary

- Optimization is a critical component of training neural networks.
- Poor optimization can lead to slow convergence, instability, or suboptimal performance.
- Key techniques include:
 - Adaptive optimization methods (Adam, RMSprop).
 - Learning rate scheduling.
 - Batch normalization.

What is Adam?

- Adam stands for **Adaptive Moment Estimation**.
- It is an optimization algorithm that combines the benefits of:
 - **Momentum**: Accelerates convergence by using moving averages of gradients.
 - **RMSprop**: Adapts learning rates based on the magnitude of recent gradients.
- Adam is widely used in deep learning due to its efficiency and robustness.

Adam: Key Components

Adam maintains two moving averages:

- **First moment (mean)**: Estimates the gradient.
- **Second moment (uncentered variance)**: Estimates the squared gradient.

Parameters:

- β_1 : Exponential decay rate for the first moment (default: 0.9).
- β_2 : Exponential decay rate for the second moment (default: 0.999).
- η : Learning rate (default: 0.001).
- ϵ : Small constant to avoid division by zero (default: 10^{-8}).

Adam: Update Rules

At each time step t :

- 1 Compute the gradient g_t .
- 2 Update the first moment (mean):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

- 3 Update the second moment (variance):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

- 4 Bias correction (to account for initialization bias):

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

- 5 Update the parameters:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Example: Optimizing a Simple Function

Consider the function:

$$f(\theta) = \theta^2$$

Goal: Minimize $f(\theta)$ using Adam.

Initialization:

- $\theta_0 = 5$ (initial parameter value).
- $\eta = 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.
- $m_0 = 0$, $v_0 = 0$.

Step 1: Compute Gradient

At $t = 1$:

- Compute the gradient:

$$g_1 = \left. \frac{\partial f}{\partial \theta} \right|_{\theta=5} = 2 \times 5 = 10$$

Step 2: Update First Moment

Update the first moment:

$$m_1 = \beta_1 m_0 + (1 - \beta_1) g_1 = 0.9 \times 0 + 0.1 \times 10 = 1$$

Bias-corrected first moment:

$$\hat{m}_1 = \frac{m_1}{1 - \beta_1^1} = \frac{1}{1 - 0.9} = 10$$

Step 3: Update Second Moment

Update the second moment:

$$v_1 = \beta_2 v_0 + (1 - \beta_2) g_1^2 = 0.999 \times 0 + 0.001 \times 100 = 0.1$$

Bias-corrected second moment:

$$\hat{v}_1 = \frac{v_1}{1 - \beta_2^1} = \frac{0.1}{1 - 0.999} = 100$$

Step 4: Update Parameter

Update the parameter:

$$\theta_1 = \theta_0 - \frac{\eta}{\sqrt{\hat{v}_1} + \epsilon} \hat{m}_1 = 5 - \frac{0.1}{\sqrt{100} + 10^{-8}} \times 10 = 5 - 0.1 = 4.9$$

Iteration 2

Repeat the process for $t = 2$:

- Compute gradient: $g_2 = 2 \times 4.9 = 9.8$.
- Update first moment: $m_2 = 0.9 \times 1 + 0.1 \times 9.8 = 1.88$.
- Update second moment: $v_2 = 0.999 \times 0.1 + 0.001 \times 96.04 = 0.195$.
- Bias correction: $\hat{m}_2 = \frac{1.88}{1-0.9^2} = 9.47$, $\hat{v}_2 = \frac{0.195}{1-0.999^2} = 97.5$.
- Update parameter: $\theta_2 = 4.9 - \frac{0.1}{\sqrt{97.5+10^{-8}}} \times 9.47 \approx 4.8$.

Summary

- Adam combines momentum and adaptive learning rates for efficient optimization.
- It maintains two moving averages: first moment (mean) and second moment (variance).
- Bias correction ensures accurate estimates, especially in early iterations.
- Adam is robust and widely used in deep learning.

RMSprop (Root Mean Square Propagation)

- Adapts the learning rate for each parameter based on the magnitude of recent gradients.
- Update rule:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t$$

where:

- v_t : Exponentially decaying average of squared gradients.
- g_t : Gradient at time t .
- Helps in stabilizing training by normalizing the learning rate.

Learning Rate Scheduling

- Adjusts the learning rate during training to improve convergence.
- Common strategies:
 - **Step Decay**: Reduce learning rate by a factor after a fixed number of epochs.
 - **Exponential Decay**: Reduce learning rate exponentially.
 - **Cosine Annealing**: Cyclically vary the learning rate.
- Example: Step Decay

$$\eta_t = \eta_0 \times \gamma^{\lfloor t/s \rfloor}$$

where:

- η_0 : Initial learning rate.
- γ : Decay factor.
- s : Step size.

Batch Normalization

- Normalizes the inputs of each layer to improve training stability and speed.
- Applied to mini-batches during training:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

where:

- μ_B : Mean of the mini-batch.
- σ_B^2 : Variance of the mini-batch.
- ϵ : Small constant for numerical stability.
- Benefits:
 - Reduces internal covariate shift.
 - Allows higher learning rates.
 - Acts as a regularizer.

- **Adam:** Combines momentum and adaptive learning rates for efficient optimization.
- **RMSprop:** Adapts learning rates based on gradient magnitudes.
- **Learning Rate Scheduling:** Dynamically adjusts learning rates to improve convergence.
- **Batch Normalization:** Normalizes layer inputs to stabilize and accelerate training.